Know Unfold

A concise codebase and presentation study guide

Rajveer Singh Jolly
African Leadership University · Mobile Application Development

Unfold Study Guide

The 20-second explanation

Unfold connects ALU students with opportunities from student-led ventures. Students discover and apply; founders publish roles and review applicants. Flutter renders the app, Riverpod owns its state, Firebase Auth identifies users, Firestore persists opportunities/applications, and security rules enforce who can perform each action.

The codebase in one picture

```
main.dart<br/>  → initializes Firebase and ProviderScope<br/>  → UnfoldApp<br/>  → AuthGate reads SessionController<br/>  → HomeShell reads feature controllers<br/>  → controllers call repository interfaces<br/>  → Firebase repository implementations read/write Firestore
```

Files you must know

File · Purpose

`lib/main.dart` · Initializes Firebase and Riverpod.

`lib/app/unfold_app.dart` · Creates `MaterialApp` and chooses `AuthGate`.

`lib/app/unfold_theme.dart` · Colors, typography, and component theme.

`lib/core/widgets/glass_surface.dart` · Reusable clipped blur/gradient/border glass component.

`lib/features/auth/session_controller.dart` · Auth repository, ALU email check, profile restore, roles, session state.

`lib/features/auth/auth_gate.dart` · Shows the correct screen for the session stage.

`lib/features/opportunities/opportunity.dart` · Opportunity model and Firestore mapping.

`lib/features/opportunities/opportunity_controller.dart` · Active feed, saved IDs, applied IDs, submission orchestration.

`lib/features/opportunities/opportunity_repository.dart` · Firestore opportunity CRUD and stream.

`lib/features/applications/application.dart` · Application model/status and mapping.

`lib/features/applications/application_review_controller.dart` · Founder stream and optimistic status updates.

`lib/features/applications/application_repository.dart` · Firestore application queries and writes.

`lib/features/cv/cv_analysis_controller.dart` · PDF selection state only; AI is not connected.

`lib/features/profile/profile_photo_controller.dart` · Validates, crops, compresses, uploads, and observes profile photos.

`lib/features/home/home_shell.dart` · Student/founder screens, navigation, search, forms and sheets.

`firestore.rules` · Backend authorization rules.

Explain Riverpod simply

The opportunity model also owns compensation truth: `isPaid`, optional `monthlyAmount`, and `currency`. The founder form makes the amount mandatory only when paid is enabled, and the discovery card/detail sheet render the same stored values.

Pull-to-refresh invalidates the opportunity, application, startup, and profile-photo providers. “Start application” opens a form; Firestore is written only after valid motivation and availability answers are submitted.

A provider is a place where a dependency or state can be accessed. A `Notifier` owns state and the methods that change it. A widget calls `ref.watch(provider)` to rebuild when that state changes. It calls `ref.read(provider.notifier)` to perform an action. Tests override providers with fake repositories, so they do not need a live Firebase connection.

Explain one complete data flow

Application submission:

1. The student completes `ApplicationFormSheet`.
2. The form validates required answers.
3. `OpportunityActivityController.submitApplication` creates an `OpportunityApplication`.
4. The UI adds the opportunity ID optimistically.
5. `FirestoreApplicationRepository.submit` writes `applications/{studentId_opportunityId}`.
6. Firestore rules verify the authenticated student, initial status, and correct opportunity owner.
7. The student application stream updates `appliedIds`.
8. If the write fails, the controller removes the optimistic ID and rethrows the error.

Firestore schema to memorize

• `users/{uid}` — name, email, role.

• `startups/{startupId}` — owner and verification status.

• `opportunities/{id}` — role data, status, `ownerId`.

• `applications/{id}` — `studentId`, `founderId`, `opportunityId`, answers, status.

• `bookmarks/{uid}/items/{opportunityId}` — private, per-user saved opportunities.

Security rules to memorize

• Profile creation requires an authenticated ALU-domain email and matching UID.

• Normal users cannot change their role after profile creation.

• Only founders create opportunities.

• Only the owning founder or admin edits/deletes an opportunity.

• Students submit applications only for themselves with `submitted` status.

• The application founder must match the opportunity owner.

• Only that founder/admin changes status; student/founder/opportunity IDs remain fixed.

• A student may withdraw only their own application.

• Bookmark paths are private to their user.

Honest feature status

Implemented

• Firebase initialization

• Google and email/password authentication

• ALU-domain gate for Google and profile creation rules

• Student/founder roles

• Opportunity feed, search, filters and details

• Opportunity repository CRUD

• Application form and submission repository

• Founder applicant review and status pipeline

• Real-time Firestore listeners

• Riverpod state management

• Glass-inspired visual system

• PDF selection/removal scaffold

• Automated tests

Partial or future

• Bookmarks persist in Firestore with optimistic updates and a local fallback.

• Seed opportunities appear when Firestore has no active records.

• Startup onboarding and founder gating are implemented; administrators verify startups in Firebase Console rather than a dedicated admin app.

• PDF upload and AI analysis are not implemented.

• Personalized matching is locked until evidence exists.

• Production App Check, notifications, pagination, and full accessibility/device QA remain future work.

Likely evaluator questions

Why Flutter?

One Dart codebase, strong custom rendering, fast iteration, mature Material widgets, and a good fit for Android delivery. Flutter's composable widgets made the reusable glass system practical.

Why Riverpod instead of `setState` everywhere?

Authentication, feeds, and applications outlive individual widgets and depend on repositories. Riverpod centralizes that state, injects dependencies, supports streams, and makes controllers testable with provider overrides. Local search text still uses widget state because it is purely presentational.

Why Firebase?

Auth and Firestore solve identity, persistence, and real-time synchronization without maintaining a custom server. Security rules place authorization beside the data.

Is hiding founder buttons secure?

No. Role-aware UI is only usability. Firestore rules enforce founder ownership even if someone calls the backend directly.

Why store applications as a top-level collection?

Both a student and a founder need to query their own applications. Top-level documents with both IDs make those queries direct and allow rules to verify both participants.

What is optimistic UI?

The screen changes immediately before the network request finishes. If Firebase succeeds, the stream confirms it. If it fails, the controller restores the previous state. This improves responsiveness without hiding

errors.

Is the CV feature AI-powered now?

No. The app currently selects/removes a PDF and models future analysis states. A production version would upload privately, call a Cloud Function, use a server-held provider key, validate structured output, and let the student edit extracted skills.

Why not put the AI API key in Flutter?

A compiled mobile app is distributed to users, so embedded secrets can be extracted. The key must stay in a trusted server environment such as Firebase Functions secret storage.

How would matching work?

AI should only turn an unstructured CV into editable structured evidence. A deterministic, explainable score can then combine skill overlap, mission alignment, learning goals, availability, and work arrangement. AI must not automatically accept or reject candidates.

What makes the UI “Liquid Glass”?

`GlassSurface` uses clipped background blur, translucent layered gradients, a luminous border/top highlight, rounded geometry, shadow depth, and Material ink response. It is an Android-appropriate interpretation, not an iOS API clone.

How would you scale it?

Add paginated indexed queries, server-maintained categories, Cloud Functions for notifications, App Check, Cloud Messaging, custom admin claims, image resizing/caching, analytics, and rule tests in the Firebase Emulator Suite.

What would you improve first?

Add a dedicated admin verification screen and emulator-tested rule suite, then build secure CV upload and editable extraction. These improve correctness before expanding the AI workflow.

Commands to remember

```
flutter pub get<br/>flutter analyze<br/>flutter test<br/>flutter run<br/>flutter build apk --debug
```

Final presentation rule

Never describe planned work as finished. The strongest answer is precise: explain what works, show the code/data flow, state the limitation, and describe the next safe implementation step.